NumPy • Pandas • Matplotlib

Python Libraries for Mathematical Foundation for Data Science

DSMC-201: Mathematical Foundation for Data Science

Course Coverage

Unit 1 — NumPy: arrays, vectorized computation, and linear algebra

Unit 2 — Pandas: structured data handling and descriptive statistics

Unit 3 — Matplotlib: visualization of data and mathematical functions

Unit 1 NumPy — Numerical Computing in Python

1.1 Introduction

NumPy ("Numerical Python") is the foundational library for numerical computing in Python. Its core object, the ndarray (n-dimensional array), stores homogeneous numerical data in contiguous memory and supports vectorized operations — whole-array computations executed without explicit Python loops. This makes NumPy dramatically faster than plain Python lists for numerical work and gives it a syntax that maps closely onto mathematical notation for vectors, matrices, and tensors.

In this course, NumPy is the computational backbone: every vector, matrix, and dataset we reason about mathematically (linear systems, transformations, statistics, probability distributions) is represented and manipulated as a NumPy array in code.

> **Convention:** *NumPy is always imported using the alias np: import numpy as np*

1.2 Creating Arrays

Arrays can be created from Python lists/tuples, or generated directly using built-in NumPy constructors.

```
import numpy as np

a = np.array([1, 2, 3])            # 1-D array (vector)
b = np.array([[1, 2], [3, 4]])     # 2-D array (matrix)

np.zeros((3, 3))                   # 3x3 matrix of zeros
np.ones((2, 4))                    # 2x4 matrix of ones
np.eye(4)                          # 4x4 identity matrix
np.full((2, 2), 7)                 # 2x2 matrix filled with 7
np.arange(0, 10, 2)                # [0 2 4 6 8]  (start, stop, step)
np.linspace(0, 1, 5)               # 5 equally spaced values in [0, 1]
```

Quick reference: array creation

Function	Purpose	Example
np.array(list)	Create an array from a Python list/tuple	np.array([1,2,3]) → array([1, 2, 3])
np.zeros(shape)	Array of zeros of the given shape	np.zeros((2,2)) → [[0. 0.], [0. 0.]]
np.ones(shape)	Array of ones of the given shape	np.ones((1,3)) → [[1. 1. 1.]]
np.eye(n)	n x n identity matrix	np.eye(3) → 3x3 identity matrix
np.full(shape, val)	Array filled with a constant value	np.full((2,2),7) → [[7 7], [7 7]]
np.arange(start, stop, step)	Evenly spaced values, like Python range()	np.arange(0,10,2) → [0 2 4 6 8]
np.linspace(start, stop, n)	n evenly spaced values over an interval (inclusive)	np.linspace(0,1,5) → [0. 0.25 0.5 0.75 1.]

1.3 Array Attributes and Data Types

```
a = np.array([[1, 2, 3], [4, 5, 6]])

a.shape     # (2, 3)  -> (rows, columns)
a.ndim      # 2       -> number of dimensions
a.size      # 6       -> total number of elements
a.dtype     # dtype('int64') -> data type of elements

a.astype(float)         # convert data type
a.reshape(3, 2)         # reshape without changing data
```

All elements of a NumPy array share a single dtype (e.g. int64, float64, bool). This uniformity is what allows NumPy to store data compactly and operate on it efficiently — unlike a Python list, which may hold mixed types.

1.4 Indexing, Slicing, and Reshaping

```
a = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])

a[0, 0]      # 10           -> single element (row 0, col 0)
a[1, :]      # [40 50 60]   -> entire row 1
a[:, 2]      # [30 60 90]   -> entire column 2
a[0:2, 0:2]  # top-left 2x2 sub-matrix

a[a > 40]    # boolean indexing -> [50 60 70 80 90]
a[[0, 2]]    # fancy indexing -> rows 0 and 2

a.flatten()  # collapse to 1-D: [10 20 30 40 50 60 70 80 90]
a.T          # transpose (swap rows and columns)
```

> **Note:** *Basic slices return views (they share memory with the original array); fancy/boolean indexing returns a copy. Modifying a view also modifies the original array.*

1.5 Vectorized Operations and Broadcasting

Arithmetic on NumPy arrays is applied element-wise, without writing loops. When array shapes differ, NumPy applies broadcasting rules to automatically expand the smaller array so the operation still makes sense — e.g. adding a single row-vector to every row of a matrix.

```
x = np.array([1, 2, 3])
y = np.array([10, 20, 30])

x + y     # [11 22 33]   element-wise addition
x * y     # [10 40 90]   element-wise multiplication
x ** 2    # [1 4 9]      element-wise power

M = np.array([[1, 2, 3], [4, 5, 6]])   # shape (2,3)
v = np.array([10, 20, 30])             # shape (3,)
M + v     # v is broadcast to every row of M

np.sqrt(x); np.exp(x); np.log(x); np.sin(x)   # universal functions (ufuncs)
```

1.6 Linear Algebra with numpy.linalg

This is where NumPy connects most directly to the mathematical foundations syllabus: vectors, matrices, systems of linear equations, and eigen-decomposition are all one function call away.

```
A = np.array([[2, 1], [1, 3]])
B = np.array([[1, 0], [0, 1]])

A @ B                  # matrix multiplication (also: np.dot(A, B))
A.T                    # transpose
np.linalg.det(A)       # determinant
np.linalg.inv(A)       # matrix inverse
np.trace(A)            # sum of diagonal elements
np.linalg.matrix_rank(A)    # rank of the matrix

eigvals, eigvecs = np.linalg.eig(A)   # eigenvalues and eigenvectors

b = np.array([1, 2])
x = np.linalg.solve(A, b)   # solves the linear system A x = b

np.linalg.norm(b)      # Euclidean (L2) norm of a vector
```

Quick reference: numpy.linalg

Function	Mathematical meaning	Example
A @ B / np.dot(A, B)	Matrix multiplication AB	[[1,2],[3,4]] @ [[2,0],[1,2]] → [[4,4],[10,8]]
A.T	Transpose A^T	[[1,2],[3,4]].T → [[1,3],[2,4]]
np.linalg.det(A)	Determinant |A|	det([[2,1],[1,3]]) → 5.0
np.linalg.inv(A)	Inverse A^{-1} (A must be square, non-singular)	inv([[2,1],[1,3]]) → [[0.6,-0.2],[-0.2,0.4]]
np.linalg.eig(A)	Eigenvalues λ and eigenvectors v such that Av = λv	eig([[2,1],[1,3]]) → λ ≈ [1.38, 3.62]
np.linalg.solve(A, b)	Solution x of the linear system Ax = b	solve([[2,1],[1,3]], [1,2]) → x ≈ [0.2, 0.6]
np.linalg.norm(v)	Vector/matrix norm (default: Euclidean / Frobenius)	norm([3,4]) → 5.0
np.linalg.matrix_rank(A)	Rank of matrix A	matrix_rank([[1,2],[2,4]]) → 1 (rows are dependent)

1.7 Statistical Functions

```
data = np.array([[85, 90, 78], [70, 88, 95], [60, 75, 80]])

np.mean(data)          # overall mean
np.mean(data, axis=0)  # column-wise mean  (axis=0 -> down the rows)
```

```
np.mean(data, axis=1)    # row-wise mean     (axis=1 -> across the columns)

np.median(data)
np.std(data)             # standard deviation
np.var(data)              # variance
np.percentile(data, 75)    # 75th percentile
np.corrcoef(x, y)         # Pearson correlation matrix between x and y
```

axis parameter: *axis=0 collapses rows (result per column); axis=1 collapses columns (result per row). This is one of the most common sources of confusion — encourage students to test both directions on a small example.*

1.8 Random Number Generation

NumPy's random module is used to simulate data, generate synthetic datasets for practice, and sample from probability distributions discussed in the theory component of the course.

```
np.random.seed(42)                # fixes the random seed for reproducibility

np.random.rand(3, 3)              # uniform random values in [0, 1), shape (3,3)
np.random.randn(1000)               # samples from the standard normal distribution
np.random.randint(1, 100, size=10)  # random integers in [1, 100)
np.random.normal(loc=50, scale=10, size=100)   # normal(mean=50, std=10)
np.random.choice([1, 2, 3, 4, 5], size=3, replace=False)   # random sample without replacement
```

1.9 Advanced Array Manipulation

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])

np.sort(a)                  # sorted copy: [1 1 2 3 4 5 6 9]
np.argsort(a)                # indices that would sort a
np.unique(a)                 # sorted unique values: [1 2 3 4 5 6 9]
np.unique(a, return_counts=True)   # unique values + their frequencies

np.where(a > 4, a, 0)         # keep values >4, else replace with 0
np.where(a > 4)              # indices where condition is True

np.clip(a, 2, 6)             # cap all values to the range [2, 6]
np.cumsum(a)                  # running (cumulative) sum
np.diff(a)                  # first differences between consecutive elements

b = np.array([1, 2, 3])
c = np.array([4, 5, 6])
np.concatenate([b, c])            # [1 2 3 4 5 6]
np.vstack([b, c])                # stack as rows -> 2x3 matrix
np.hstack([b, c])                # stack side by side -> [1 2 3 4 5 6]
np.split(np.arange(9), 3)         # split into 3 equal parts
```

np.where as a vectorized if-else: *np.where(condition, a, b) is the array equivalent of writing "for each element: a if condition else b" — without an explicit Python loop. Extremely useful for recoding/labelling data.*

1.10 Advanced Linear Algebra

Beyond determinants and simple solves, NumPy provides matrix decompositions that appear later in the course — in dimensionality reduction (SVD/PCA), regression (least squares), and numerical stability of solving systems.

```
A = np.array([[4, 2], [2, 3]])

U, S, Vt = np.linalg.svd(A)       # Singular Value Decomposition: A = U . diag(S) . Vt
Q, R = np.linalg.qr(A)            # QR decomposition: A = Q . R  (Q orthogonal, R upper-triangular)
L = np.linalg.cholesky(A)         # Cholesky decomposition (A must be symmetric positive-definite): A = L . L.T

np.linalg.pinv(A)                 # Moore-Penrose pseudo-inverse (works even for non-square/singular A)

# Least-squares fit of a straight line y = mx + c to noisy data
x = np.array([0, 1, 2, 3])
y = np.array([1, 3, 5, 7])
M = np.vstack([x, np.ones_like(x)]).T
m, c = np.linalg.lstsq(M, y, rcond=None)[0]

# einsum: a compact notation for sums-of-products (dot products, traces, batched matmul, ...)
np.einsum('ij,jk->ik', A, A)      # equivalent to A @ A
```

1.11 Vectorizing Custom Functions & Curve Fitting

```
def is_even(n):
    return n % 2 == 0

vec_is_even = np.vectorize(is_even)   # apply a scalar Python function element-wise
vec_is_even(np.array([1, 2, 3, 4]))   # [False  True False  True]

# Fit a degree-2 polynomial (least-squares) to data points
x = np.array([0, 1, 2, 3, 4])
y = np.array([1, 3, 7, 13, 21])
coeffs = np.polyfit(x, y, deg=2)      # [a, b, c] for a*x^2 + b*x + c
poly = np.poly1d(coeffs)              # callable polynomial: poly(2.5)
```

Performance note: *np.vectorize is a convenience wrapper for readability, not a genuine performance optimization — under the hood it still loops in Python. For real speed-ups, prefer expressing the computation directly with array operations and ufuncs.*

Quick reference: advanced NumPy

Function	Purpose	Example
np.sort(a) / np.argsort(a)	Sort values / get sorting indices	np.sort([3,1,2]) → [1 2 3]
np.unique(a)	Distinct values (optionally with counts)	np.unique([1,1,2,3,3]) → [1 2 3]

np.where(cond, a, b)	Vectorized conditional selection	np.where([1,-2,3]>0,[1,-2,3],0) → [1 0 3]
np.clip(a, lo, hi)	Cap values to a range	np.clip([1,5,10],2,8) → [2 5 8]
np.cumsum(a) / np.diff(a)	Running total / consecutive differences	np.cumsum([1,2,3]) → [1 3 6]
np.concatenate / vstack / hstack	Combine arrays along an axis	np.hstack([[1,2],[3,4]]) → [1 2 3 4]
np.linalg.svd(A)	Singular Value Decomposition	svd([[3,0],[0,2]]) → singular values [3, 2]
np.linalg.qr(A)	QR decomposition	qr([[1,0],[0,1]]) → Q = I, R = I
np.linalg.cholesky(A)	Cholesky decomposition (SPD matrices)	chol([[4,2],[2,3]]) → L = [[2,0],[1,1.41]]
np.linalg.pinv(A)	Pseudo-inverse (non-square / singular A)	pinv([[1,2],[3,4],[5,6]]) → 2x3 pseudo-inverse
np.linalg.lstsq(A, b)	Least-squares solution to Ax ≈ b	fit y=mx+c to (0,1),(1,3),(2,5) → m=2, c=1
np.polyfit(x, y, deg)	Least-squares polynomial fit	polyfit([0,1,2],[1,3,7],2) → [a,b,c] coefficients
np.einsum(subscripts, ...)	General tensor contraction notation	einsum('ij,jk->ik',A,B) → same as A @ B

Unit 2 Pandas — Structured Data Handling

2.1 Introduction

Pandas is built on top of NumPy and provides two core labelled data structures: the Series (1-D, like a single column) and the DataFrame (2-D, like a spreadsheet or a database table). Where NumPy is ideal for pure numerical arrays, Pandas is designed for real-world, heterogeneous, and often messy datasets — the kind students will encounter as CSV files, survey responses, or exported spreadsheets.

> **Convention:** *Pandas is always imported using the alias pd: import pandas as pd*

2.2 Series and DataFrame

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])   # 1-D labelled array

df = pd.DataFrame({
   'Name':  ['Aarav', 'Bhavna', 'Chirag'],
   'Marks': [78, 91, 65],
   'Attendance': [88, 95, 72]
})

df.shape       # (3, 3)  -> (rows, columns)
df.columns     # Index(['Name', 'Marks', 'Attendance'])
df.index       # RangeIndex(start=0, stop=3, step=1)
df.dtypes      # data type of each column
df.info()      # summary: columns, non-null counts, dtypes, memory
df.describe()  # count, mean, std, min, quartiles, max for numeric columns
```

2.3 Reading and Writing Data

```
df = pd.read_csv('students.csv')
df = pd.read_excel('students.xlsx', sheet_name='Sheet1')

df.head()         # first 5 rows
df.tail(3)        # last 3 rows

df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', index=False)
```

2.4 Indexing and Selection

```
df['Marks']               # a single column -> Series
df[['Name', 'Marks']]     # multiple columns -> DataFrame

df.loc[0, 'Marks']        # label-based access: row label 0, column 'Marks'
df.loc[0:2, ['Name','Marks']]

df.iloc[0, 1]             # position-based access: row 0, column 1
```

```
df.iloc[0:2, 0:2]

df[df['Marks'] > 75]                    # conditional (boolean) filtering
df[(df['Marks'] > 75) & (df['Attendance'] > 85)]   # multiple conditions
```

loc vs iloc: *loc selects by label (row/column names); iloc selects by integer position, exactly like list indexing. This distinction matters once a DataFrame's index is not the default 0, 1, 2, … (e.g. after filtering or setting a custom index).*

2.5 Handling Missing Data

```
df.isnull()            # boolean mask of missing values
df.isnull().sum()      # count of missing values per column

df.dropna()            # drop rows containing any missing value
df.dropna(axis=1)      # drop columns containing any missing value

df.fillna(0)                    # replace missing values with 0
df.fillna(df.mean(numeric_only=True))   # replace with column mean
df['Marks'].fillna(df['Marks'].median(), inplace=True)
```

2.6 Descriptive Statistics

```
df['Marks'].mean()
df['Marks'].median()
df['Marks'].std()         # standard deviation
df['Marks'].var()         # variance
df.corr(numeric_only=True) # pairwise Pearson correlation matrix
df.cov(numeric_only=True)  # pairwise covariance matrix
df.describe()             # full statistical summary in one call
```

The correlation matrix from df.corr() is a direct, code-level counterpart of the correlation coefficient covered in the statistics component of the course, and is often the first step before visualizing relationships between variables in Matplotlib.

2.7 GroupBy and Aggregation

groupby() implements the “split – apply – combine” pattern: split the data into groups based on some key, apply a function to each group independently, then combine the results.

```
df.groupby('Subject')['Marks'].mean()         # average marks per subject

df.groupby('Subject').agg({
   'Marks': 'mean',
   'Attendance': 'sum'
})

df.groupby('Subject').size()         # number of records per group
df.groupby('Subject')['Marks'].agg(['mean', 'std', 'max', 'min'])
```

2.8 Merging, Joining, and Concatenation

```
pd.concat([df1, df2])              # stack rows (same columns)
pd.concat([df1, df2], axis=1)      # stack columns side by side

pd.merge(df1, df2, on='Roll No', how='inner')   # SQL-style join
# how = 'inner' | 'left' | 'right' | 'outer'
```

pd.merge() join types

how	Result	Example
inner	Keep only keys present in both DataFrames	keys {1,2,3} ∩ {2,3,4} → rows for 2, 3 only
left	Keep all keys from the left DataFrame	all rows from df1; unmatched df2 columns → NaN
right	Keep all keys from the right DataFrame	all rows from df2; unmatched df1 columns → NaN
outer	Keep all keys from either DataFrame (fill gaps with NaN)	keys {1,2,3} ∪ {2,3,4} → rows for 1,2,3,4

2.9 Applying Custom Functions

```
df['Marks'].apply(lambda x: x + 5)          # apply a function to every element of a Series
df.apply(lambda col: col.max() - col.min())  # apply a function to every column of a DataFrame

df['Grade'] = df['Marks'].map({80: 'A', 60: 'B'})   # element-wise mapping/replacement (Series only)

df.pipe(lambda d: d.assign(Total=d['Marks'] + d['Attendance']))   # chain a custom transformation
```

apply vs. vectorized ops: *apply() is flexible but runs a Python function per row/column, so it is slower than a built-in vectorized operation. Prefer direct arithmetic (df['Marks'] * 1.1) or np.where() whenever possible, and reserve apply() for logic that genuinely cannot be vectorized.*

2.10 Reshaping: pivot_table, melt, and crosstab

```
# Wide summary table: average marks by Subject and Term
pd.pivot_table(df, values='Marks', index='Subject', columns='Term', aggfunc='mean')

# Long <-> wide conversions
wide_df.melt(id_vars=['Name'], var_name='Subject', value_name='Marks')   # wide -> long
long_df.pivot(index='Name', columns='Subject', values='Marks')           # long -> wide

# Frequency cross-tabulation between two categorical columns
pd.crosstab(df['Department'], df['Bonus Eligible?'])
```

2.11 Binning, Ranking, and Categorical Data

```
pd.cut(df['Marks'], bins=[0, 45, 60, 75, 85, 100],
```

```
    labels=['F', 'C', 'B', 'A', 'O'])          # bin into fixed-width / custom ranges

pd.qcut(df['Marks'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])   # bin into equal-frequency quartiles

df['Marks'].rank(ascending=False)     # rank students (1 = highest mark)

df['Department'] = df['Department'].astype('category')   # categorical dtype: saves memory, speeds up groupby
```

2.12 Window Functions: rolling, expanding, shift

Window functions compute a statistic over a sliding or growing window of rows — the standard tool for moving averages and other trend-smoothing calculations on ordered/time-series data.

```
df['Marks'].rolling(window=3).mean()     # 3-row moving average
df['Marks'].expanding().mean()          # cumulative (running) average from the start
df['Marks'].shift(1)                    # shift values down by one row (previous value)
df['Marks'].pct_change()                # percentage change from the previous row
```

2.13 Working with Dates and Strings

```
df['Date'] = pd.to_datetime(df['Date'])        # parse a column into datetime objects
df['Year'] = df['Date'].dt.year                # .dt accessor exposes date components
df['Month'] = df['Date'].dt.month_name()

df.set_index('Date').resample('M')['Marks'].mean()   # resample a time series to monthly averages

df['Name'].str.upper()              # .str accessor: vectorized string operations
df['Name'].str.contains('Sharma')
df['Name'].str.split(' ', expand=True)   # split into separate columns
```

2.14 Fast Filtering with query()

```
df.query('Marks > 75 and Attendance > 85')   # readable alternative to boolean-mask filtering

threshold = 75
df.query('Marks > @threshold')               # reference a Python variable with @
```

Quick reference: advanced Pandas

Function	Purpose	Example
df.apply(func)	Apply a custom function along rows/columns	df['Marks'].apply(lambda x: x+5) → adds 5 to every mark
pd.pivot_table(...)	Excel-style pivot summary	pivot_table(df,'Marks','Subject','Term','mean') → mean marks grid
df.melt(...) / df.pivot(...)	Convert between long and wide format	melt(id_vars=['Name']) → one row per Name-Subject pair
pd.crosstab(a, b)	Frequency table of two categorical	crosstab(Dept, Bonus) → count of Yes/No

	variables	per department
pd.cut / pd.qcut	Bin continuous data into discrete categories	cut([40,65,90],[0,60,80,100]) → ['low','mid','high']
df.rank()	Rank values within a column	rank([70,90,80]) → [3, 1, 2] (1 = highest by default)
df.rolling(w).mean()	Moving-window statistic (e.g. moving average)	rolling(3).mean() on [1,2,3,4] → [NaN,NaN,2.0,3.0]
pd.to_datetime() / .dt	Parse and decompose dates	to_datetime('2026-08-19').dt.year → 2026
df.resample(rule)	Aggregate a time series to a new frequency	resample('M').mean() → one average value per month
.str accessor	Vectorized string operations on a text column	df['Name'].str.upper() → 'AARAV SHARMA'
df.query('expr')	Filter rows using a readable expression string	df.query('Marks > 75') → same as df[df.Marks > 75]

Unit 3 Matplotlib — Data Visualization

3.1 Introduction

Matplotlib is Python's foundational plotting library. It is used throughout this course to visualize mathematical functions (e.g. curves, probability distributions) and to visually explore datasets produced with NumPy and Pandas — turning a table of numbers into a picture that reveals trends, spread, and relationships at a glance.

> **Convention:** *The pyplot interface is imported using the alias plt: import matplotlib.pyplot as plt*

3.2 Basic Plot Types

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y)                # line plot -> functions, trends over a continuous variable
plt.scatter(x, y)             # scatter plot -> relationship between two variables
plt.bar(['A','B','C'], [10, 25, 15])   # bar chart -> comparing categories
plt.hist(np.random.randn(1000), bins=20)   # histogram -> distribution / frequency
plt.boxplot(np.random.randn(100))          # box plot -> median, quartiles, outliers
plt.pie([30, 45, 25], labels=['A','B','C'])  # pie chart -> proportion of a whole

plt.show()
```

Choosing the right plot type

Function	Best used for	Example
plt.plot(x, y)	Continuous functions, time series, trends	plt.plot(x, x**2) → draws the parabola y = x^2
plt.scatter(x, y)	Relationship / correlation between two variables	plt.scatter(Marks, Attendance) → one dot per student
plt.bar(x, height)	Comparing values across discrete categories	plt.bar(['DBMS','OS','CN'], [78,91,65]) → 3 bars
plt.hist(data, bins=n)	Distribution / frequency of a single variable	plt.hist(marks_array, bins=10) → marks frequency chart
plt.boxplot(data)	Median, quartiles, spread, and outliers	plt.boxplot([55,60,65,70,95]) → shows 95 as an outlier
plt.pie(values)	Proportions that make up a whole	plt.pie([30,45,25], labels=['A','B','C']) → 3 pie slices

3.3 Customizing Plots

```
plt.figure(figsize=(8, 5))          # set figure size (width, height in inches)

plt.plot(x, y, color='blue', linestyle='--', marker='o', label='sin(x)')

plt.title('Sine Function')
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.legend()
plt.grid(True)

plt.savefig('sine_plot.png', dpi=300)   # save the figure to a file
plt.show()
```

Good practice: *Every plot shown or submitted in a lab report should have a title, labelled axes, and (when more than one series is plotted) a legend. Marks are typically lost for unlabeled charts, so make this a habit early.*

3.4 Subplots and Figure Layout

The object-oriented interface (fig, ax = plt.subplots(...)) is the recommended way to build multi-panel figures, since it gives explicit control over each subplot (axes object) rather than relying on Matplotlib's implicit "current axes" state.

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))    # 1 row, 2 columns of subplots

ax[0].plot(x, np.sin(x))
ax[0].set_title('sin(x)')

ax[1].plot(x, np.cos(x), color='orange')
ax[1].set_title('cos(x)')

plt.tight_layout()   # avoid overlapping titles/labels
plt.show()
```

3.5 Statistical Visualizations

Matplotlib is commonly combined with NumPy's random sampling and Pandas' summary statistics to visualize the theoretical concepts covered in the mathematical foundation syllabus — probability distributions, correlation structure, and comparative summaries across groups.

```
# Distribution of a normally-distributed sample
data = np.random.normal(loc=50, scale=10, size=1000)
plt.hist(data, bins=30, color='skyblue', edgecolor='black')
plt.title('Distribution of Sample Data (mean=50, std=10)')
plt.xlabel('Value'); plt.ylabel('Frequency')
plt.show()

# Correlation heatmap from a Pandas DataFrame
corr = df.corr(numeric_only=True)
plt.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)
```

```
plt.colorbar(label='Correlation coefficient')
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title('Correlation Heatmap')
plt.show()
```

3.6 Twin Axes, Annotations, and Fill Regions

```
fig, ax1 = plt.subplots()

ax1.plot(x, np.sin(x), color='blue')
ax1.set_ylabel('sin(x)', color='blue')

ax2 = ax1.twinx()                    # share the x-axis, independent y-axis
ax2.plot(x, np.exp(x), color='red')
ax2.set_ylabel('exp(x)', color='red')

plt.annotate('local max', xy=(1.57, 1), xytext=(3, 0.8),
             arrowprops=dict(arrowstyle='->'))     # point to a specific feature

plt.fill_between(x, np.sin(x), 0, alpha=0.3)   # shade the area under a curve
```

When to use twinx(): *Use a secondary y-axis only when two series share the same x-axis but have very different scales/units (e.g. Revenue in ₹ vs. Profit Margin in %) — exactly the combo-chart pattern used in the Excel practicals.*

3.7 Error Bars and Contour / 3-D Plots

```
means = [10, 15, 12]
errors = [1.2, 0.8, 1.5]
plt.errorbar(['A', 'B', 'C'], means, yerr=errors, fmt='o', capsize=5)   # mean +/- uncertainty

# Contour plot of a bivariate function z = f(x, y)
x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(X) * np.cos(Y)
plt.contourf(X, Y, Z, cmap='viridis')
plt.colorbar()

# Basic 3-D surface plot
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis')
```

3.8 Complex Layouts with GridSpec

```
fig = plt.figure(figsize=(8, 6))
gs = fig.add_gridspec(2, 2)            # 2x2 grid of cells to combine flexibly
```

```
ax1 = fig.add_subplot(gs[0, :])        # top row, spans both columns
ax2 = fig.add_subplot(gs[1, 0])         # bottom-left cell
ax3 = fig.add_subplot(gs[1, 1])         # bottom-right cell
```

3.9 Styling and Log Scales

```
plt.style.use('seaborn-v0_8-whitegrid')   # apply a built-in style theme to all subsequent plots
print(plt.style.available)                # list all available style names

plt.yscale('log')     # logarithmic y-axis -> useful for data spanning several orders of magnitude
plt.xscale('log')
```

Quick reference: advanced Matplotlib

Function	Purpose	Example
ax.twinx()	Secondary y-axis sharing the same x-axis	ax2 = ax1.twinx() → plot Revenue and Margin% together
plt.annotate(text, xy, xytext)	Label a specific point with an arrow	annotate('peak', xy=(1.57,1), xytext=(3,0.8))
plt.fill_between(x, y1, y2)	Shade the region between two curves	fill_between(x, sin(x), 0) → shades area under sin(x)
plt.errorbar(x, y, yerr)	Plot values with uncertainty/error bars	errorbar(['A','B'], [10,15], yerr=[1,2]) → bars with caps
plt.contourf(X, Y, Z)	Filled contour plot of a bivariate function	contourf(X, Y, sin(X)*cos(Y)) → colour-filled contours
ax.plot_surface(X, Y, Z)	3-D surface plot (mpl_toolkits.mplot3d)	plot_surface(X, Y, X**2+Y**2) → 3-D bowl shape
fig.add_gridspec(rows, cols)	Flexible, unevenly spanning subplot layouts	gs[0,:] → one subplot spanning the whole top row
plt.style.use(name)	Apply a predefined visual theme	plt.style.use('ggplot') → all plots adopt ggplot styling
plt.xscale/yscale('log')	Logarithmic axis scaling	plt.yscale('log') → compresses [1, 10, 100, 1000, ...]

Quick-Reference Summary

A condensed cheat-sheet of the most frequently used functions across the three libraries, for quick lookup during lab sessions and assessments.

NumPy vs. Pandas — equivalent operations

Task	NumPy	Pandas
Create structure	np.array(list)	pd.DataFrame(dict)
Inspect shape	a.shape → (3,4)	df.shape → (3,4)
Select subset	a[1:3, 0:2]	df.loc[1:2, ['Name','Marks']]
Filter by condition	a[a > 5]	df[df['Marks'] > 75]
Mean / std / var	np.mean(a) → 75.0	df['Marks'].mean() → 75.0
Handle missing values	np.nan, np.isnan(a)	df.fillna(df.mean())
Combine datasets	np.concatenate([a, b])	pd.concat([df1, df2])

Matplotlib — task to function

Goal	Matplotlib call	Example
Plot a function / trend	plt.plot(x, y)	plt.plot(x, np.exp(x)) → exponential growth curve
Compare two variables	plt.scatter(x, y)	plt.scatter(Attendance, Marks) → correlation pattern
Compare categories	plt.bar(categories, values)	plt.bar(['E','F','H'], [203,135,110]) → dept. KRA totals
Show a distribution	plt.hist(data, bins=n)	plt.hist(np.random.randn(1000), bins=30) → bell curve
Show spread & outliers	plt.boxplot(data)	plt.boxplot(marks_list) → quartile box with whiskers
Multiple panels	fig, ax = plt.subplots(rows, cols)	subplots(1,2) → sin(x) and cos(x) side by side

Suggested workflow: *A typical lab exercise flows as: load/clean data with Pandas -> compute numerical/statistical quantities with NumPy or Pandas -> visualize the result with Matplotlib. Encouraging students to see this pipeline explicitly helps them understand why all three libraries are taught together.*